

COMP 3311 数据库管理系统—课程总结 (Part 2: Lectures 8–11 —SQL)

课程: COMP 3311 Database Management Systems 时间: 2026 年 6 月 20 日–26 日 覆盖: Lecture 8 –Lecture 11 (SQL DML & DDL) 配套 Part 1: Lectures 1–7 (概念建模、关系设计、范式化、关系代数)

参考数据库模式 (Bank Schema):

Account(accountNo, balance, branchName)
Borrower(clientId, loanNo)
Branch(branchName, district, liabilities, assets)
Client(clientId, name, hkid, address, district, rating)
Depositor(clientId, accountNo)
Loan(loanNo, amount, year, branchName)
Tags(clientId, tag)

目录

1. Lecture 8: SQL DML —基本查询结构
2. Lecture 9: SQL DML —聚合与子查询
3. Lecture 10: SQL DML —分析函数与数据修改
4. Lecture 11: SQL DML & DDL —过程化 SQL 与模式定义

8. Lecture 8: SQL DML —基本查询结构

8.1 SQL 概述

SQL 由两部分组成: - DDL (数据定义语言): 定义关系模式 - DML (数据操作语言): 查询和修改数据

SQL 查询的基本形式对应关系代数 $\pi_A(\bowtie_P(R_1 \times R_2 \times \dots \times R_n))$:

```
SELECT A1, A2, ..., An
FROM R1, R2, ..., Rn
WHERE P;
```

关键性质: SQL 查询结果是关系 (但可能包含重复), 所以查询可以组合/嵌套。

8.2 SELECT 子句 (投影)

功能	语法	说明
基本投影	SELECT a ₁ , a ₂ FROM R	选择指定列
所有属性	SELECT * FROM R	返回所有列

功能	语法	说明
去重	SELECT DISTINCT a FROM R	默认不去重 ，需显式使用 DISTINCT
保留重复	SELECT ALL a FROM R	等同于省略 ALL (默认行为)
算术运算	SELECT a*100 FROM R	可包含 +, -, *, /
空值替换	SELECT COALESCE(a, 'unknown')	返回第一个非 null 参数

8.3 WHERE 子句 (选择)

比较运算符

= 等于	> 大于	< 小于
>= 大于等于	<= 小于等于	<> 或 != 不等于

特殊运算符

运算符	语法	说明
BETWEEN	WHERE rating BETWEEN 7 AND 9	范围查询 (包含端点)
NOT BETWEEN	WHERE rating NOT BETWEEN 7 AND 9	范围排除
AND/OR/NOT	WHERE a=1 AND b=2	布尔运算 (AND 优先级 高于 OR)
IS NULL	WHERE district IS NULL	不能用 = NULL (null 不匹配任何值)
IS NOT NULL	WHERE district IS NOT NULL	非空判断
LIKE	WHERE addr LIKE '%Main%'	字符串模式匹配: % = 任 意子串, _ = 单个字符
ESCAPE	WHERE s LIKE '20\%%' ESCAPE '\'	转义特殊字符
REGEXP_LIKE	WHERE REGEXP_LIKE(name, '^Ste(v ph)en')	正则表达式匹配 (Oracle)

重要: null 比较的陷阱—WHERE district = null **永远不会**匹配任何行!

8.4 FROM 子句 (连接)

连接类型	SQL 语法	说明
笛卡尔积	FROM R ₁ , R ₂ 或 FROM R ₁ CROSS JOIN R ₂	所有组合
自然连接	FROM R ₁ NATURAL JOIN R ₂	所有同名属性等值连接, 公共属性只出现一次
USING 连接	FROM R ₁ JOIN R ₂ USING (a)	指定公共属性连接
-连接 / 等值连接	FROM R ₁ JOIN R ₂ ON R ₁ .a=R ₂ .b	用 ON 指定任意连接条件
WHERE 连接	FROM R ₁ , R ₂ WHERE R ₁ .a=R ₂ .b	等价于等值连接 (传统写法)
左外连接	FROM R ₁ LEFT OUTER JOIN R ₂ ON ...	R 中不匹配的元组保留 (填充 null)
右外连接	FROM R ₁ RIGHT OUTER JOIN R ₂ ON ...	R 中不匹配的元组保留
全外连接	FROM R ₁ FULL OUTER JOIN R ₂ ON ...	双方不匹配的都保留

注意事项: - 自然连接不能用 NATURAL + USING 一两者互斥 - 自然连接中**不能用**表名限定公共属性 (如 R.a) - 外连接的连接条件**只能**在 ON 中指定, **不能**在 WHERE 中 - 当属性名有歧义时, **必须**用表名限定 (如 Loan.loanNo)

8.5 集合操作

-- 并 (去重)

```
SELECT clientId FROM Depositor UNION SELECT clientId FROM Borrower;
```

-- 交 (去重)

```
SELECT clientId FROM Depositor INTERSECT SELECT clientId FROM Borrower;
```

-- 差 (去重) — Oracle 用 MINUS, SQL 标准用 EXCEPT

```
SELECT clientId FROM Depositor MINUS SELECT clientId FROM Borrower;
```

- 默认**自动去重**; 加 ALL 保留所有重复 (如 UNION ALL)
- 要求**并兼容** (同属性数 + 同类型)

8.6 重命名

-- 属性重命名

```
SELECT clientId, loanNo AS loanId FROM Borrower;
```

-- 关系重命名 (别名 / correlation name)

```
SELECT B.loanNo FROM Borrower B, Loan L WHERE B.loanNo = L.loanNo;
```

- Oracle 中别名的 AS 关键字在 FROM 子句中**不允许**

- **自连接**必须用别名来区分同一个表的不同实例

8.7 ORDER BY —排序

```
SELECT name, rating FROM Client ORDER BY rating DESC, name ASC;
```

- ASC 默认升序, DESC 降序
- null 值默认是**最大值** (升序在最后, 降序在最前)
- 可用 NULLS FIRST / NULLS LAST 调整

8.8 FETCH —限制结果行数 (Oracle)

```
-- 前 3 行
```

```
SELECT * FROM Client ORDER BY rating DESC FETCH FIRST 3 ROWS ONLY;
```

```
-- 前 3 行 (含并列)
```

```
SELECT * FROM Client ORDER BY rating DESC FETCH FIRST 3 ROWS WITH TIES;
```

```
-- 跳过 2 行后取 1 行 (第三高)
```

```
SELECT DISTINCT rating FROM Client ORDER BY rating DESC
  OFFSET 2 ROWS FETCH NEXT 1 ROW ONLY;
```

通用语法: [OFFSET n ROWS] FETCH [FIRST | NEXT] [n ROWS | PERCENT n]
[ONLY | WITH TIES]

8.9 CASE 语句

```
SELECT name,
  CASE WHEN rating <= 3 THEN 'high risk'
        WHEN rating <= 7 THEN 'medium risk'
        WHEN rating <= 10 THEN 'low risk'
        ELSE 'unknown risk'
  END AS riskCategory
FROM Client;
```

- 返回第一个匹配 WHEN 的值, 无匹配时返回 ELSE (或 null)

9. Lecture 9: SQL DML —聚合与子查询

9.1 聚合函数

函数	说明	注意事项
COUNT(*)	统计元组数	包含 null
COUNT(a)	统计非 null 值数	忽略 null
COUNT(DISTINCT a)	统计唯一非 null 值数	
SUM(a)	求和	忽略 null, 只能是数字

函数	说明	注意事项
AVG(a)	平均值	忽略 null, 只能是数字
MAX(a) / MIN(a)	最大/最小值	忽略 null
MEDIAN(a)	中位数	Oracle
STDEV(a)	标准差	忽略 null

重要规则: - 除 COUNT(*) 外, 所有聚合函数忽略 null - 空集合的聚合返回 null, 除了 COUNT 返回 0 - COUNT(DISTINCT *) 是非法语法

9.2 GROUP BY 一分组聚合

```
SELECT branchName, COUNT(*), AVG(balance)
FROM Account
GROUP BY branchName;
```

关键规则: - SELECT 中的非聚合属性必须出现在 GROUP BY 中 - GROUP BY 中的属性不必出现在 SELECT 中

概念执行顺序: FROM → WHERE → GROUP BY (形成分组) → HAVING → SELECT → ORDER BY

9.3 HAVING 一分组过滤

```
SELECT branchName, AVG(balance) as avgBal
FROM Account
GROUP BY branchName
HAVING AVG(balance) > 55000;
```

- HAVING 只能与 GROUP BY 一起使用
- HAVING 中的条件在分组之后评估
- WHERE 在分组之前过滤元组; HAVING 在分组之后过滤分组

SQL 查询评估顺序:

1. FROM → 获取关系
2. WHERE → 过滤元组 (分组前)
3. GROUP BY → 形成分组
4. HAVING → 过滤分组
5. SELECT → 计算聚合, 选择输出
6. ORDER BY → 排序

注意: SELECT 中定义的别名不能被 WHERE/GROUP BY/HAVING 使用, 因为它们评估更早。

检测重复: 利用 GROUP BY + HAVING COUNT(*) 判断存在性/唯一性。

-- 只有一个账户的客户

```
SELECT clientId FROM Depositor d JOIN Account a ON d.accountNo=a.accountNo
WHERE branchName='Star House'
```

```
GROUP BY clientId HAVING COUNT(*) = 1;
```

```
-- 至少两个账户的客户
... HAVING COUNT(*) > 1;
```

9.4 嵌套子查询

核心思想：子查询返回关系，因此可以嵌套在任何需要值或集合的地方。

```
-- 比较运算符 + 标量子查询（必须返回单值）
SELECT * FROM Loan WHERE amount > (SELECT AVG(amount) FROM Loan);
```

```
-- 获取第三高评级的所有客户
SELECT name FROM Client
WHERE rating = (SELECT DISTINCT rating FROM Client
                ORDER BY rating DESC NULLS LAST
                OFFSET 2 ROWS FETCH NEXT 1 ROW ONLY);
```

9.5 集成员测试

```
-- IN: 是否在集合中
SELECT clientId FROM Borrower
WHERE clientId IN (SELECT clientId FROM Depositor);
```

```
-- NOT IN: 是否不在集合中
SELECT clientId FROM Borrower
WHERE clientId NOT IN (SELECT clientId FROM Depositor);
```

集合对集合比较：

```
-- 找到每个分行的最高余额客户
SELECT branchName, clientId, name, accountNo, balance
FROM Client NATURAL JOIN Depositor NATURAL JOIN Account
WHERE (branchName, balance) IN
      (SELECT branchName, MAX(balance) FROM Account GROUP BY branchName);
```

9.6 SOME / ALL

操作	含义	等价
> SOME(set)	大于集合中至少一个	> MIN(set)
< SOME(set)	小于集合中至少一个	< MAX(set)
= SOME(set)	等于集合中至少一个	等价于 IN
<> SOME(set)	不等于集合中至少一个	不等价于 NOT IN
> ALL(set)	大于集合中所有值	> MAX(set)
< ALL(set)	小于集合中所有值	< MIN(set)
= ALL(set)	等于集合中所有值	不等价于 IN
<> ALL(set)	不等于集合中所有值	等价于 NOT IN

```
-- 资产大于 Yau Tsim Mong 所有分行的分行
SELECT branchName FROM Branch
WHERE assets > ALL (SELECT assets FROM Branch WHERE district='Yau Tsim Mong');
```

9.7 EXISTS / NOT EXISTS

```
-- 同时有贷款和账户的客户 (相关子查询 / correlated subquery)
SELECT clientId FROM Depositor D
WHERE EXISTS (SELECT * FROM Borrower B WHERE D.clientId = B.clientId);

-- 有账户但没有贷款的客户
SELECT clientId FROM Depositor D
WHERE NOT EXISTS (SELECT * FROM Borrower B WHERE D.clientId = B.clientId);
```

- NOT EXISTS 可以模拟**集合包含**: A 包含 B NOT EXISTS (B MINUS A)

作用域规则: 子查询中可引用外层别名 (如 D.clientId); 内层别名不能在外层使用。

9.8 FROM 子句中的子查询

```
SELECT branchName, avgBalance
FROM (SELECT branchName, AVG(balance) AS avgBalance
      FROM Account GROUP BY branchName) result
WHERE avgBalance > (SELECT AVG(balance) FROM Account);
```

- 结果被称为**派生关系** (derived/temporary relation), 查询结束后丢弃
- Oracle 的作用域限制可能需要使用 WITH 子句代替

9.9 WITH 子句 (CTE)

```
WITH result (branchName, avgBalance) AS
  (SELECT branchName, AVG(balance) FROM Account GROUP BY branchName)
SELECT branchName, avgBalance
FROM result
WHERE avgBalance = (SELECT MAX(avgBalance) FROM result);
```

- 定义**只在当前查询中可见**的临时关系
- 比 FROM 子查询更清晰, 解决了 Oracle 的作用域限制

10. Lecture 10: SQL DML —分析函数与数据修改

10.1 聚合函数 vs 分析函数

特性	聚合函数 (Aggregate)	分析函数 (Analytic)
返回行数	每组一行	每个输入行一行
需要 GROUP BY	是 (多行时)	不需要

特性	聚合函数 (Aggregate)	分析函数 (Analytic)
可以分组	GROUP BY	PARTITION BY
出现位置	SELECT, HAVING	仅 SELECT 和 ORDER BY
执行顺序	—	最后执行 (ORDER BY 之前)

-- 聚合: 每组一行 (非法 —没有 GROUP BY 但有非聚合属性)

```
SELECT accountNo, balance, SUM(balance) FROM Account; -- ❖
```

-- 分析: 每个 account 行都显示合计值

```
SELECT accountNo, balance, SUM(balance) OVER () AS totalBalance FROM Account; -- ❖
```

10.2 分析函数语法

```
analytic_function([args]) OVER (
  [PARTITION BY clause]
  [ORDER BY clause]
  [windowing_clause]
)
```

子句	作用
PARTITION BY	将查询结果分区 (类似 GROUP BY, 但不合并行)
ORDER BY	指定分区内的排序
windowing_clause	指定滑动窗口 (见 10.7)

10.3 RANK / DENSE_RANK

-- 按资产排名

```
SELECT branchName, assets,
  RANK() OVER (ORDER BY assets DESC) AS branchRank
FROM Branch;
```

函数	并列处理	是否有“空缺”
RANK()	相同值同排名	有 (1, 2, 2, 4, ...)
DENSE_RANK()	相同值同排名	无 (1, 2, 2, 3, ...)
ROW_NUMBER()	并列随机分配	无, 始终唯一
PERCENT_RANK()	$(rank-1)/(n-1)$ 分数形式	
CUME_DIST()	p/n 累积分布	

Top-N 查询: 将排序查询嵌套后过滤排名。

```
SELECT branchName, assets FROM
  (SELECT branchName, assets, RANK() OVER (ORDER BY assets DESC) r FROM Branch)
WHERE r <= 3;
```


10.4 NTILE

将元组**均匀分配**到 n 个桶中（用于百分位数分析）。

```
SELECT branchName, assets,
       NTILE(4) OVER (ORDER BY assets DESC) AS quartile
FROM Branch;
-- 如果总数不能被  $n$  整除, 各桶最多差 1
```

10.5 LISTAGG

将多行数据连接为一个分隔列表。

```
-- 聚合模式: 每个 district 一行, 客户名列表
SELECT district, LISTAGG(name, ', ') WITHIN GROUP (ORDER BY name) AS clients
FROM Client GROUP BY district;

-- 分析模式: 每个 loan 行都显示同一分行的所有金额列表
SELECT loanNo, amount, branchName,
       LISTAGG(amount, ', ') WITHIN GROUP (ORDER BY amount)
       OVER (PARTITION BY branchName) AS "all loan amounts"
FROM Loan WHERE year='2025';
```

10.6 窗口 (Windowing)

概念: 在分析函数的分区内定义一个**滑动窗口**, 每个元组基于其窗口内的数据计算函数值。窗口随当前元组移动。

```
SELECT year,
       AVG(yearLoanTotal) OVER (
         ORDER BY year
         ROWS BETWEEN UNBOUNDED PRECEDING AND 1 FOLLOWING
       ) AS movingAvg
FROM (SELECT year, SUM(amount) AS yearLoanTotal FROM Loan GROUP BY year);
```

窗口语法:

ROWS | RANGE BETWEEN <start> AND <end>

边界	含义
UNBOUNDED PRECEDING	分区第一行
UNBOUNDED FOLLOWING	分区最后一行
CURRENT ROW	当前行/值
n PRECEDING	前 n 行 (ROWS) 或前 n 范围 (RANGE)
n FOLLOWING	后 n 行 (ROWS) 或后 n 范围 (RANGE)

区别	ROWS	RANGE
单位	物理行 (行数)	逻辑值 (值偏移)
相同值处理	每行独立窗口	相同排序值的行共享窗口

10.7 数据修改

INSERT —插入元组

-- 单行插入 (按属性顺序)

```
INSERT INTO Account VALUES ('A-332', 1200, 'Pacific Place');
```

-- 指定属性插入 (推荐, 顺序无关)

```
INSERT INTO Account (accountNo, branchName, balance) VALUES ('A-334', 'Pacific Place', 1200);
```

-- 从查询结果批量插入

```
INSERT INTO Account
  SELECT loanNo, 200, branchName FROM Loan WHERE branchName='Pacific Place';
```

注意: 从查询插入时**不能用** VALUES 关键字。

DELETE —删除元组

-- 条件删除

```
DELETE FROM Account WHERE branchName='Pacific Place';
```

-- 全表删除

```
DELETE FROM Account; -- 删除所有元组!
```

-- 复杂删除 (引用子查询)

```
DELETE FROM Depositor
WHERE accountNo IN (SELECT accountNo FROM Depositor NATURAL JOIN Account
                     WHERE branchName='Star House');
```

UPDATE —更新元组

-- 条件更新

```
UPDATE Account SET balance = 50000 WHERE accountNo = 'A-333';
```

-- 批量更新

```
UPDATE Account SET balance = balance * 1.05 WHERE balance < 10000;
```

-- CASE 合并多次更新

```
UPDATE Account SET balance = CASE
  WHEN balance <= 10000 THEN balance * 1.05
  ELSE balance * 1.06
END;
```

用 CASE 替代多条 UPDATE 语句，避免**执行顺序**问题。

11. Lecture 11: SQL DML & DDL —过程化 SQL 与模式定义

11.1 DBMS API 与 PL/SQL

DBMS API: 客户端应用通过网络与 DBMS 通信 (如 Oracle 使用 port 1521)。

Oracle PL/SQL: 类似 C 的过程式编程语言, SQL 语句嵌入式使用。- **过程 (Procedure)**: 不返回值, 用 exec 调用 - **函数 (Function)**: 用 RETURN 返回值

PL/SQL 基本结构

```
CREATE OR REPLACE PROCEDURE proc_name [ AS | IS ]
-- 声明段 (变量、类型、游标)
BEGIN
-- 执行段 (必须存在)
-- 允许: SELECT, INSERT, UPDATE, DELETE
-- 不允许: CREATE, DROP, ALTER, RENAME
EXCEPTION
-- 异常处理段
END;
```

内容摘要

- **数据类型**: NUMBER, INT, CHAR, VARCHAR2 等; 也可用 table.column%TYPE 或 table%ROWTYPE
- **控制流**: IF-THEN-ELSIF, CASE, LOOP, WHILE, FOR, EXIT, CONTINUE, GOTO
- **SELECT INTO**: 将查询结果赋值给程序变量, **必须只返回一行**, 否则触发异常

```
SELECT name, rating INTO clientName, clientRating
FROM Client WHERE clientId = cid;
-- 多于一行 → TOO_MANY_ROWS; 零行 → NO_DATA_FOUND
```

11.2 游标 (Cursor)

游标用于在 PL/SQL 中**逐行**处理多行查询结果。

```
-- 声明
CURSOR cursor_name IS select_statement;

-- 隐式迭代 (推荐)
FOR record IN cursor_name LOOP
-- record.column_name 访问当前行的值
END LOOP;

-- 显式管理
```

```

OPEN cursor_name;
FETCH cursor_name INTO variables;
CLOSE cursor_name;

```

游标状态: %FOUND, %NOTFOUND, %ISOPEN, %ROWCOUNT

11.3 异常处理

```

EXCEPTION
  WHEN NO_DATA_FOUND THEN ... -- SELECT INTO 返回零行
  WHEN TOO_MANY_ROWS THEN ... -- SELECT INTO 返回多行
  WHEN DUP_VAL_ON_INDEX THEN ... -- 主键重复
  WHEN OTHERS THEN ... -- 任何其他异常

```

可自定义异常: exception_name EXCEPTION; → RAISE exception_name;

11.4 SQL DDL —数据定义语言

语句	用途
CREATE TABLE	创建关系模式
ALTER TABLE ... ADD	添加属性
ALTER TABLE ... DROP COLUMN	删除属性
DROP TABLE	删除关系 (模式 + 数据)

基本域类型

类型	说明
CHAR(n)	定长字符串 (空格填充)
VARCHAR2(n)	变长字符串 (Oracle 推荐; 标准用 VARCHAR)
INT	整数
NUMBER(p,d)	定点数 (p= 总位数, d= 小数位)
FLOAT(n)	浮点数
DATE	日期 (Oracle 包含时间)
TIMESTAMP	日期 + 时间

11.5 完整性约束 (Integrity Constraints)

```

CREATE TABLE Client (
  clientId INT PRIMARY KEY,           -- 主键 (自动 NOT NULL)
  name VARCHAR2(15) NOT NULL,        -- 非空
  hkid CHAR(10) NOT NULL UNIQUE,     -- 候选键 (可为 null)
  address VARCHAR2(20) NOT NULL,
  district VARCHAR2(10) NOT NULL,
  rating INT,

```

```

    UNIQUE (hkid)                                -- 表级候选键
);

```

约束	关键字	说明
NOT NULL	NOT NULL	属性不能为 null
PRIMARY KEY	PRIMARY KEY	主键：唯一 + 非空
UNIQUE	UNIQUE	候选键：唯一（可为 null）
FOREIGN KEY	REFERENCES T(k)	外键：值必须匹配被引用表的主键或为 null
CHECK	CHECK (P)	自定义谓词条件

11.6 外键操作 (Referential Actions)

```

FOREIGN KEY (accountNo) REFERENCES Account(accountNo)
ON DELETE CASCADE      -- 删除被引用元组时，级联删除引用元组
-- 或 ON DELETE SET NULL  -- 删除被引用元组时，将外键设为 null
-- 或 ON DELETE SET DEFAULT -- 删除被引用元组时，将外键设为默认值
-- 或无（默认）          -- 禁止删除被引用的元组

```

完整选项： - ON DELETE: CASCADE / SET NULL / SET DEFAULT / (默认拒绝)
 - ON UPDATE: CASCADE (**Oracle 不支持**) / (默认拒绝)

限制： 如果外键是主键的一部分，不能使用 SET NULL 或 SET DEFAULT。

11.7 CHECK 约束

```

CREATE TABLE Loan (
    loanNo CHAR(5) PRIMARY KEY,
    amount NUMBER(8,2) CHECK (amount >= 1000 AND amount <= 100000),
    -- 或 CHECK (amount BETWEEN 1000 AND 100000)
    year CHAR(4),
    branchName VARCHAR2(15) NOT NULL
);

```

11.8 视图 (Views)

视图 = 通过查询定义的“虚拟关系”，用于隐藏数据。

```

-- 创建视图
CREATE VIEW BranchLoan AS
    SELECT loanNo, year, branchName FROM Loan; -- 隐藏 amount

-- 查询视图 (如普通表)
SELECT loanNo FROM BranchLoan WHERE branchName='Star House';

-- 删除视图
DROP VIEW BranchLoan;

```

重要：视图**绝不应该**用来表达查询！视图是用来控制数据访问的。

可更新视图的条件（所有条件都必须满足）：1. FROM 子句只包含一个关系 2. SELECT 只包含属性名（无表达式、聚合、DISTINCT） 3. 未列出的属性可以设为 null 4. 查询没有 GROUP BY 或 HAVING

11.9 断言 (Assertions)

```
CREATE ASSERTION loanSumConstraint AS CHECK
(NOT EXISTS
  (SELECT * FROM Branch
    WHERE (SELECT SUM(amount) FROM Loan NATURAL JOIN Branch)
          >=
          (SELECT SUM(balance) FROM Account NATURAL JOIN Branch)));
```

- 断言可能涉及多个关系，在任何更新时检查
- 开销很大，应谨慎使用
- Oracle 不支持断言

11.10 触发器 (Triggers)

触发器由数据库修改事件**自动执行**，用于实现其他约束无法表达的完整性规则。

```
CREATE OR REPLACE TRIGGER overdraft
BEFORE UPDATE OF balance ON Account
FOR EACH ROW
WHEN (NEW.balance < 0)
DECLARE
  currentYear Loan.year%TYPE;
BEGIN
  SELECT TO_CHAR(SYSDATE, 'YYYY') INTO currentYear FROM DUAL;
  INSERT INTO Loan VALUES (:OLD.accountNo, -:NEW.balance, currentYear, :OLD.branchName);
  INSERT INTO Borrower (SELECT clientName, accountNo FROM Depositor
                        WHERE accountNo = :OLD.accountNo);
  :NEW.balance := 0;
END;
```

触发器语法要素：

要素	说明
BEFORE AFTER	触发时机
DELETE INSERT UPDATE OF col	触发事件
FOR EACH ROW	逐行触发（省略则语句级触发一次）
WHEN (condition)	触发条件
:OLD.col / :NEW.col	修改前/后的值（在 PL/SQL 中需加 :）

用途：实现跨表约束、审计日志、自动维护聚合值（如分行负债总额）、业务规则自动化。

SQL 核心知识地图

SQL 查询基础 (L8)

- ◆◆◆ SELECT-FROM-WHERE 结构
- ◆◆◆ 投影: DISTINCT, 算术运算, COALESCE
- ◆◆◆ 选择: 比较, BETWEEN, LIKE, REGEXP_LIKE, NULL
- ◆◆◆ 连接: NATURAL JOIN, JOIN ON/USING, OUTER JOIN
- ◆◆◆ 集合: UNION, INTERSECT, EXCEPT/MINUS
- ◆◆◆ 重命名: AS (列), 别名 (表)
- ◆◆◆ 排序限制: ORDER BY, FETCH/OFFSET
- ◆◆◆ 条件逻辑: CASE

聚合与子查询 (L9)

- ◆◆◆ 聚合函数: COUNT, SUM, AVG, MAX, MIN
- ◆◆◆ GROUP BY + HAVING
- ◆◆◆ 嵌套子查询
- ◆◆◆ 集合成员: IN, NOT IN
- ◆◆◆ 集合比较: SOME, ALL
- ◆◆◆ 存在性: EXISTS, NOT EXISTS
- ◆◆◆ FROM 子查询 (派生表)
- ◆◆◆ WITH 子句 (CTE)

分析函数与数据修改 (L10)

- ◆◆◆ 分析函数 vs 聚合函数
- ◆◆◆ RANK, DENSE_RANK, ROW_NUMBER
- ◆◆◆ NTILE (分桶)
- ◆◆◆ LISTAGG (列表聚合)
- ◆◆◆ 窗口: ROWS/RANGE BETWEEN
- ◆◆◆ INSERT / DELETE / UPDATE
- ◆◆◆ CASE 合并更新

DDL 与过程化 SQL (L11)

- ◆◆◆ PL/SQL: 过程, 函数, 块结构
 - ◆◆◆ 游标: 显式/隐式, FOR LOOP
 - ◆◆◆ 异常处理
 - ◆◆◆ DDL: CREATE/ALTER/DROP TABLE
 - ◆◆◆ 域类型: CHAR, VARCHAR2, NUMBER, DATE, TIMESTAMP
 - ◆◆◆ 约束: PK, FK, UNIQUE, CHECK, NOT NULL
 - ◆◆◆ 外键操作: CASCADE, SET NULL, SET DEFAULT
 - ◆◆◆ 视图: 创建/查询/更新条件
 - ◆◆◆ 断言 (Oracle 不支持)
 - ◆◆◆ 触发器: BEFORE/AFTER, FOR EACH ROW, :OLD/:NEW
-

课程参考: “Fundamentals of Database Systems” (教材章节标注于各 Lecture 幻灯中) **关联文件:** COMP3311_Lecture_Summary_Part1.md
—E-R 模型、关系设计、范式化、关系代数